

# Object Relationships in System Design

Association, Aggregation, and Composition

1

## Association

Independent objects interact

2

## Aggregation

Weak ownership relationship

3

## Composition

Strong ownership relationship

*A foundational framework for flexible, maintainable, and scalable software architecture*

📄 N. Murali Sai

192372309

# Introduction to Object Relationships

**Object relationships** define how classes interact and depend on each other in OOP systems. The relationship you choose affects coupling, ownership, and how easily your system can change.

## Why They Matter

- Enable flexible, maintainable software architecture
- Reduce bugs by clarifying ownership and lifecycle
- Allow teams to work on independent modules in parallel

### Association

Independent objects  
interact

### Aggregation

Weak "has-a" ownership



# Association: Independent Objects Interacting

**Definition:** Two objects interact with each other but remain fully independent. Neither object owns the other, and each can exist without the other. This is the most flexible of the three relationship types.

## Relationship Multiplicity

### One-to-One

Teacher ↔ Student

### One-to-Many

Customer ↔ Orders

### Many-to-Many

Student ↔ Courses

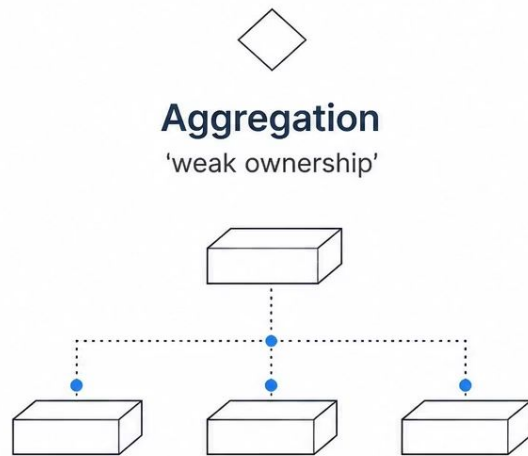
## Key Advantages

- **Loose Coupling**  
Objects can exist and evolve independently without breaking each other
- **Better Communication**  
Clear, defined interaction points between objects across the system
- **Easy to Maintain**  
Modify one object without cascading side effects on its associates

A **Teacher** can exist without **Students**, and **Students** can exist without a specific Teacher — this independence is what makes association so powerful for resilient system design.

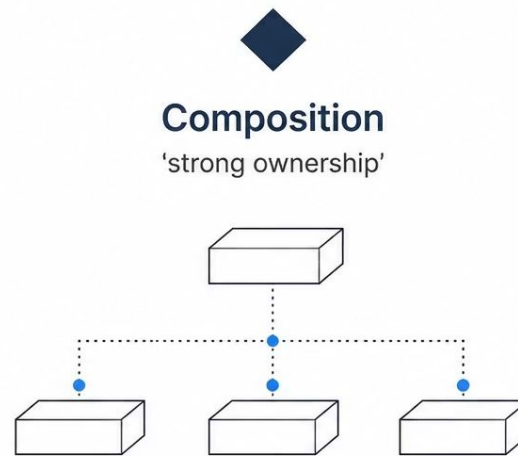
# Aggregation vs. Composition

Both are "has-a" relationships, but they differ in **lifecycle dependency**. This determines what happens to child objects when the parent is destroyed.



child can exist independently of parent

example: Department has Employees —  
employees can change departments and  
survive if department is deleted



child cannot exist without parent,  
parent controls child lifecycle

example: House has Rooms —  
rooms are destroyed when the  
house is destroyed

## Aggregation — Hollow Diamond ◇

- Child can outlive the parent
- Weak "has-a" ownership
- Example: Department → Employees

## Composition — Filled Diamond ◆

- Child is destroyed with the parent
- Strong ownership and lifecycle control
- Example: House → Rooms

# Impact on Software Architecture

Choosing the right relationship type has direct, measurable consequences on code quality, team productivity, and system longevity. These are not abstract principles — they affect every pull request, every refactor, and every new feature.



## Code Reusability

Well-defined relationships enable components to be reused across projects without modification or tight coupling to a specific context.



## Flexibility

Loose coupling allows systems to adapt to changing requirements without triggering cascading rewrites across the codebase.



## Modularity

Clear relationships create independent, testable modules that teams can develop and deploy in parallel.



## Scalability

Systems grow without cascading changes — new features integrate cleanly when relationship boundaries are well defined.

- ❏ **Low coupling, high cohesion:** Objects focus on their single responsibility while minimizing dependencies on other objects — the gold standard of software architecture.

# Real-World Applications & Key Takeaways

## Real-World Domain Examples

| Domain     | Composition              | Association        |
|------------|--------------------------|--------------------|
| E-Commerce | Order → LineItems        | Customer ↔ Product |
| Library    | Book → Chapters          | Member ↔ Book      |
| Hospital   | Patient → Medical Record | Doctor ↔ Patient   |
| Banking    | Account → Transactions   | Customer ↔ Account |

## The Architect's Decision Guide

01

### Use Association

When objects need to interact but must remain fully independent of each other's lifecycle

02

### Use Aggregation

When a parent "has" children, but those children can meaningfully exist on their own

03

### Use Composition

When children are intrinsic parts of the parent and have no identity without it

**Key Takeaway:** These three relationship types are tools in your architectural toolkit. The right choice — made early — is what separates systems that scale gracefully from systems that become maintenance nightmares.